```
-- excersize 1
CREATE TABLE Customers (
    customer_id NUMBER PRIMARY KEY,
    name VARCHAR2(50),
    age NUMBER,
    balance NUMBER,
    isVIP VARCHAR2(5)
);

CREATE TABLE Loans (
    loan_id NUMBER PRIMARY KEY,
    customer_id NUMBER,
    interest_rate NUMBER,
    due_date DATE,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

SELECT table_name FROM user_tables;

INSERT INTO Customers VALUES (1, 'Raj', 65, 15000, 'FALSE');
INSERT INTO Customers VALUES (2, 'Amit', 45, 8000, 'FALSE');
INSERT INTO Customers VALUES (3, 'Sneha', 70, 20000, 'FALSE');

INSERT INTO Loans VALUES (101, 1, 10, SYSDATE + 10);
INSERT INTO Loans VALUES (102, 2, 12, SYSDATE + 40);
INSERT INTO Loans VALUES (103, 3, 9, SYSDATE + 20);

COMMIT;


SELECT * FROM Customers;
SELECT * FROM Loans;


SET SERVEROUTPUT ON;

BEGIN
    FOR rec IN (SELECT customer_id, age FROM Customers) LOOP
        IF rec.age > 60 THEN
            UPDATE Loans
            SET interest_rate = interest_rate - 1
            WHERE customer_id = rec.customer_id;
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

```
BEGIN
   FOR rec IN (SELECT customer_id, balance FROM Customers) LOOP
      IF rec.balance > 10000 THEN
         UPDATE Customers
         SET isVIP = 'TRUE'
         WHERE customer_id = rec.customer_id;
      END IF;
   END LOOP;
   COMMIT;
END;
/

BEGIN
   FOR rec IN (
      SELECT c.name, l.due_date
      FROM Customers c
      JOIN Loans l ON c.customer_id = l.customer_id
      WHERE l.due_date BETWEEN SYSDATE AND SYSDATE + 30
   ) LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Reminder: Dear ' || rec.name ||
         ', your loan is due on ' || rec.due_date
      );
   END LOOP;
END;
/

-- excersize 2
CREATE TABLE Accounts (
   acc_id NUMBER PRIMARY KEY,
   name VARCHAR2(50),
   balance NUMBER
);

INSERT INTO Accounts VALUES (1, 'Raj', 5000);
INSERT INTO Accounts VALUES (2, 'Amit', 3000);
COMMIT;

CREATE TABLE Employees (
   emp_id NUMBER PRIMARY KEY,
   name VARCHAR2(50),
   salary NUMBER
);

INSERT INTO Employees VALUES (101, 'John', 20000);
INSERT INTO Employees VALUES (102, 'Alice', 25000);
COMMIT;
```

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
   from_acc NUMBER,
   to_acc NUMBER,
   amount NUMBER
)
IS
   v_balance NUMBER;
BEGIN
   -- Check balance
   SELECT balance INTO v_balance
   FROM Accounts
   WHERE acc_id = from_acc;

   IF v_balance < amount THEN
      RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');
   END IF;

   -- Deduct
   UPDATE Accounts
   SET balance = balance - amount
   WHERE acc_id = from_acc;

   -- Add
   UPDATE Accounts
   SET balance = balance + amount
   WHERE acc_id = to_acc;

   COMMIT;

EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
      ROLLBACK;
END;
/

BEGIN
   SafeTransferFunds(1, 2, 2000);
END;
/


CREATE OR REPLACE PROCEDURE UpdateSalary(
   empid NUMBER,
   percent NUMBER
)
IS
BEGIN
```

```
    UPDATE Employees
    SET salary = salary + (salary * percent / 100)
    WHERE emp_id = empid;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Employee not found');
    END IF;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        ROLLBACK;
END;
/

BEGIN
    UpdateSalary(101, 10);
END;
/

CREATE OR REPLACE PROCEDURE AddNewCustomer(
    cid NUMBER,
    cname VARCHAR2,
    cage NUMBER,
    cbalance NUMBER
)
IS
BEGIN
    INSERT INTO Customers
    VALUES (cid, cname, cage, cbalance, 'FALSE');

    COMMIT;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: Customer ID already exists');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        ROLLBACK;
END;
/

BEGIN
    AddNewCustomer(1, 'Test', 30, 5000);
END;
/
```

```
-- excersize 3
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
   UPDATE Accounts
   SET balance = balance + (balance * 0.01);

   COMMIT;
END;
/

BEGIN
   ProcessMonthlyInterest;
END;
/

SELECT * FROM Accounts;
SELECT * FROM employees;

CREATE TABLE Employeeswithdp (
   emp_id NUMBER PRIMARY KEY,
   name VARCHAR2(50),
   department VARCHAR2(50),
   salary NUMBER
);
INSERT INTO employeeswithdp VALUES (1, 'John', 'IT', 30000);
INSERT INTO employeeswithdp VALUES (2, 'Alice', 'HR', 25000);
INSERT INTO employeeswithdp VALUES (3, 'Bob', 'IT', 28000);
COMMIT;
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
   dept VARCHAR2,
   bonus_percent NUMBER
)
IS
BEGIN
   UPDATE employeeswithdp
   SET salary = salary + (salary * bonus_percent / 100)
   WHERE department = dept;

   COMMIT;
END;
/

BEGIN
   UpdateEmployeeBonus('IT', 10);
END;
/
```

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    from_acc NUMBER,
    to_acc NUMBER,
    amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    -- Check balance
    SELECT balance INTO v_balance
    FROM Accounts
    WHERE acc_id = from_acc;

    IF v_balance < amount THEN
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
        RETURN;
    END IF;

    -- Deduct
    UPDATE Accounts
    SET balance = balance - amount
    WHERE acc_id = from_acc;

    -- Add
    UPDATE Accounts
    SET balance = balance + amount
    WHERE acc_id = to_acc;

    COMMIT;
END;
/

BEGIN
    TransferFunds(1, 2, 2000);
END;
/

CREATE OR REPLACE FUNCTION CalculateAge(
    dob DATE
)
RETURN NUMBER
IS
    age NUMBER;
BEGIN
    age := FLOOR(MONTHS_BETWEEN(SYSDATE, dob) / 12);
    RETURN age;
END;
```

```
/


--excersize 4
CREATE OR REPLACE FUNCTION CalculateAge(
   dob DATE
)
RETURN NUMBER
IS
   age NUMBER;
BEGIN
   age := FLOOR(MONTHS_BETWEEN(SYSDATE, dob) / 12);
   RETURN age;
END;
/

SELECT CalculateAge(DATE '2005-06-01') AS Age FROM dual;


CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
   p_amount NUMBER,
   p_rate NUMBER,
   p_years NUMBER
)
RETURN NUMBER
IS
   r NUMBER;
   n NUMBER;
   emi NUMBER;
BEGIN
   r := (p_rate / 100) / 12;
   n := p_years * 12;

   emi := (p_amount * r * POWER(1 + r, n)) /
          (POWER(1 + r, n) - 1);

   RETURN emi;
END;
/
SELECT CalculateMonthlyInstallment(100000, 10, 2) AS EMI FROM dual;

CREATE OR REPLACE FUNCTION HasSufficientBalance(
   p_acc_id NUMBER,
   p_amount NUMBER
)
RETURN VARCHAR2
IS
   v_balance NUMBER;
```

```
BEGIN
   SELECT balance INTO v_balance
   FROM Accounts
   WHERE acc_id = p_acc_id;

   IF v_balance >= p_amount THEN
      RETURN 'YES';
   ELSE
      RETURN 'NO';
   END IF;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 'ACCOUNT NOT FOUND';
   WHEN TOO_MANY_ROWS THEN
      RETURN 'DATA ERROR';
END;
/
SELECT HasSufficientBalance(1, 2000) FROM dual;

ALTER TABLE Customers ADD LastModified DATE;
-- excerise 5
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
   :NEW.LastModified := SYSDATE;
END;
/
UPDATE Customers
SET name = 'Raj Updated'
WHERE customer_id = 1;

SELECT * FROM Customers;

CREATE TABLE Transactions (
   trans_id NUMBER PRIMARY KEY,
   acc_id NUMBER,
   amount NUMBER,
   type VARCHAR2(10)
);

CREATE TABLE AuditLog (
   log_id NUMBER GENERATED ALWAYS AS IDENTITY,
   trans_id NUMBER,
   action_date DATE
);
```

```
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
   INSERT INTO AuditLog (trans_id, action_date)
   VALUES (:NEW.trans_id, SYSDATE);
END;
/

INSERT INTO Transactions VALUES (1, 1, 500, 'DEPOSIT');

SELECT * FROM AuditLog;

CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
   v_balance NUMBER;
BEGIN
   -- Deposit check
   IF :NEW.type = 'DEPOSIT' AND :NEW.amount <= 0 THEN
      RAISE_APPLICATION_ERROR(-20003, 'Deposit must be positive');
   END IF;

   -- Withdrawal check
   IF :NEW.type = 'WITHDRAW' THEN
      SELECT balance INTO v_balance
      FROM Accounts
      WHERE acc_id = :NEW.acc_id;

      IF v_balance < :NEW.amount THEN
         RAISE_APPLICATION_ERROR(-20004, 'Insufficient Balance');
      END IF;
   END IF;

END;
/

-- wrong case -INSERT INTO Transactions VALUES (2, 1, -100, 'DEPOSIT');

INSERT INTO Transactions VALUES (4, 1, 500, 'WITHDRAW');

--excerise 6

DECLARE
   CURSOR trans_cursor IS
      SELECT c.name, t.amount, t.type, t.trans_id
      FROM Customers c
```

```
    JOIN Transactions t ON c.customer_id = t.acc_id
    WHERE EXTRACT(MONTH FROM SYSDATE) = EXTRACT(MONTH FROM
SYSDATE);

  v_name Customers.name%TYPE;
  v_amount Transactions.amount%TYPE;
  v_type Transactions.type%TYPE;
  v_id Transactions.trans_id%TYPE;

BEGIN
  OPEN trans_cursor;

  LOOP
    FETCH trans_cursor INTO v_name, v_amount, v_type, v_id;
    EXIT WHEN trans_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      'Customer: ' || v_name ||
      ', Transaction: ' || v_type ||
      ', Amount: ' || v_amount
    );
  END LOOP;

  CLOSE trans_cursor;
END;
/

DECLARE
  CURSOR acc_cursor IS
    SELECT acc_id FROM Accounts;

  v_acc_id Accounts.acc_id%TYPE;

BEGIN
  OPEN acc_cursor;

  LOOP
    FETCH acc_cursor INTO v_acc_id;
    EXIT WHEN acc_cursor%NOTFOUND;

    UPDATE Accounts
    SET balance = balance - 500
    WHERE acc_id = v_acc_id;

  END LOOP;

  CLOSE acc_cursor;
```

```
    COMMIT;
END;
/

DECLARE
    CURSOR loan_cursor IS
        SELECT loan_id, interest_rate FROM Loans;

    v_loan_id Loans.loan_id%TYPE;
    v_rate Loans.interest_rate%TYPE;

BEGIN
    OPEN loan_cursor;

    LOOP
        FETCH loan_cursor INTO v_loan_id, v_rate;
        EXIT WHEN loan_cursor%NOTFOUND;

        UPDATE Loans
        SET interest_rate = v_rate + 1
        WHERE loan_id = v_loan_id;

    END LOOP;

    CLOSE loan_cursor;

    COMMIT;
END;
/

– excersize 7

CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer(cid NUMBER, cname VARCHAR2, cage NUMBER,
cbalance NUMBER);
    PROCEDURE UpdateCustomer(cid NUMBER, cname VARCHAR2);
    FUNCTION GetBalance(cid NUMBER) RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(cid NUMBER, cname VARCHAR2, cage NUMBER,
cbalance NUMBER) IS
    BEGIN
        INSERT INTO Customers (customer_id, name, age, balance, isVIP)
        VALUES (cid, cname, cage, cbalance, 'FALSE');
    END;
```

```
    PROCEDURE UpdateCustomer(cid NUMBER, cname VARCHAR2) IS
    BEGIN
        UPDATE Customers
        SET name = cname
        WHERE customer_id = cid;
    END;

    FUNCTION GetBalance(cid NUMBER) RETURN NUMBER IS
        v_balance NUMBER;
    BEGIN
        SELECT balance INTO v_balance
        FROM Customers
        WHERE customer_id = cid;
        RETURN v_balance;
    END;

END CustomerManagement;
/

BEGIN
    CustomerManagement.AddCustomer(20, 'TestUser', 25, 7000);
END;
/

CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee(eid NUMBER, ename VARCHAR2, dept VARCHAR2, sal
NUMBER);
    PROCEDURE UpdateEmployee(eid NUMBER, sal NUMBER);
    FUNCTION AnnualSalary(eid NUMBER) RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee(eid NUMBER, ename VARCHAR2, dept VARCHAR2, sal
NUMBER) IS
    BEGIN
        INSERT INTO Employees (emp_id, name, salary)
        VALUES (eid, ename, sal);
    END;

    PROCEDURE UpdateEmployee(eid NUMBER, sal NUMBER) IS
    BEGIN
        UPDATE Employees
        SET salary = sal
        WHERE emp_id = eid;
    END;
```

```
    FUNCTION AnnualSalary(eid NUMBER) RETURN NUMBER IS
        v_salary NUMBER;
    BEGIN
        SELECT salary INTO v_salary
        FROM Employees
        WHERE emp_id = eid;
        RETURN v_salary * 12;
    END;

END EmployeeManagement;
/

DESC Employees;

BEGIN
    EmployeeManagement.HireEmployee(200, 'TestEmp', 'IT', 30000);
END;
/

CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount(aid NUMBER, name VARCHAR2, bal NUMBER);
    PROCEDURE CloseAccount(aid NUMBER);
    FUNCTION TotalBalance(cid NUMBER) RETURN NUMBER;
END AccountOperations;
/

ALTER TABLE Accounts ADD customer_id NUMBER;
CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount(aid NUMBER, name VARCHAR2, bal NUMBER) IS
    BEGIN
        INSERT INTO Accounts (acc_id, name, balance)
        VALUES (aid, name, bal);
    END;

    PROCEDURE CloseAccount(aid NUMBER) IS
    BEGIN
        DELETE FROM Accounts WHERE acc_id = aid;
    END;

    FUNCTION TotalBalance(cid NUMBER) RETURN NUMBER IS
        total NUMBER;
    BEGIN
        SELECT SUM(balance) INTO total
        FROM Accounts
        WHERE customer_id = cid;
        RETURN total;
```

```
    END;

END AccountOperations;
/

BEGIN
    AccountOperations.OpenAccount(10, 'Raj', 5000);
END;
/
```